## Introduction

Subarray sum problems show up constantly in coding interviews because they test something deeper than syntax — they test whether you actually understand how sums accumulate and cancel. **Minimum Positive Sum Subarray** (LeetCode 3364) looks simple on the surface: find a contiguous slice of an array, within a length range, whose sum is the smallest positive number possible. But the moment you try to solve it efficiently, it exposes a classic trap — the instinct to reach for a sliding window, which quietly falls apart the second negative numbers enter the picture.

This problem is worth learning not because you'll be asked this exact LeetCode number in an interview, but because the pattern underneath it — **prefix sums turned into a sorted-lookup problem** — appears in dozens of subarray problems. Once you see it here, you'll recognize it everywhere.

## Problem Overview

You're given an integer array `nums` and two integers, `l` and `r`. You need to find a contiguous subarray whose length is somewhere between `l` and `r` (inclusive), such that its sum is strictly greater than zero, and as small as possible among all such subarrays.

If no subarray in that length range has a positive sum, return `-1`.

In plain terms: you're not looking for the biggest win, and you're not looking for just any win — you're looking for the *smallest possible* win, among all the window sizes you're allowed to use.

## Example

Take `nums = [3, -2, 1, 4]`, `l = 2`, `r = 3`.

Every valid window (length 2 or 3):

| Window (indices) | Elements | Sum |
|---|---|---|
| 0–1 | `[3, -2]` | 1 |
| 1–2 | `[-2, 1]` | -1 |
| 2–3 | `[1, 4]` | 5 |
| 0–2 | `[3, -2, 1]` | 2 |
| 1–3 | `[-2, 1, 4]` | 3 |

Sums that qualify (strictly positive): 1, 5, 2, 3. The smallest among them is **1**, from the window `[3, -2]`.

Notice window `[-2, 1]` sums to -1 — it's thrown out entirely, no matter how "small" the magnitude is. Only positive sums are candidates.

## Intuition

The naive instinct is: "just grab the smallest positive number in the array." That fails immediately — a longer stretch can cancel down to something smaller than any single element while staying positive. So length alone doesn't tell you which subarray wins; you have to actually consider sums.

The second instinct, especially if you've solved "maximum subarray" problems before, is to reach for a **sliding window**: grow or shrink the window and track the best sum as you go. This is where the problem quietly punishes you. Sliding window logic relies on a monotonic relationship — shrinking the window should move the sum predictably in one direction. With negative numbers in the mix, growing a window can make the sum *smaller*, not bigger, and shrinking it can make the sum worse. The monotonic assumption that makes sliding window algorithms work simply isn't true here.

So we need a different lens. Build a **prefix sum array**: `prefix[i]` is the sum of the first `i` elements. Now any subarray sum is just one prefix value minus an earlier one: `sum(i..j) = prefix[j+1] - prefix[i]`.

Reframe the goal: for each ending point, you want the **largest prefix value that's still smaller than today's prefix** — because that gives you the smallest positive gap. That's no longer a sliding-window problem; it's a lookup problem. And if you keep the valid prefixes sorted, finding "the closest smaller value" is a binary search, not a scan.

Picture a shelf of prefix totals, sorted, where only the totals that keep your window length between `l` and `r` are allowed to sit on the shelf at any time. Each step, you add one new prefix and retire one that's aged out, then binary search the shelf for the closest value below today's prefix.

## Brute Force Solution

**Idea:** For every starting index, try every valid ending index within the length bounds, compute the sum directly, and track the smallest positive one found.

**Algorithm:** 1. For each start index `i`. 2. For each length from `l` to `r` (bounded by array size). 3. Sum the window directly (or use a running total as you extend). 4. If the sum is positive, compare

against the best found so far.

**Advantages:** Dead simple to write and reason about; no risk of subtle bugs with sorted structures or binary search bounds.

**Disadvantages:** Recomputing or extending sums for every start/length pair means you're doing roughly `O(n²)` work in the worst case — fine for `n ≤ 100` as in this problem's constraints, but it won't scale to thousands of elements.

**Time complexity:** `O(n²)` (or `O(n² · (r-l))` depending on how sums are computed). **Space complexity:** `O(1)` beyond the input.

## Optimal Solution

The optimal approach has three moving pieces:

1. **Prefix sums** — reduce every subarray sum to a difference of two prefix values.
2. **A sorted window of valid prefixes** — at each step `end`, the only prefixes allowed in the window are those that keep the subarray length between `l` and `r`. That means prefix index `end - r` through `end - l` are valid starts.
3. **Binary search** — instead of scanning the sorted window for the closest smaller prefix, use `bisect_left` to jump straight to the insertion point, then look at the entry just before it.

Walking through the process for each `end` from `l` to `n`: - Insert the prefix that just became old enough to be a valid start (`prefix[end - l]`). - Remove the prefix that's now too old to be a valid start (anything representing a window longer than `r`). - Binary search the sorted window for where `prefix[end]` would land. - If there's an entry immediately before that position, `prefix[end] - that entry` is a valid candidate — keep the smallest one seen.

This converts an `O(n)` scan per step into an `O(log n)` lookup per step — the part that actually makes the algorithm fast, at least for the search itself (more on insertion cost below).

## Python Code

```
from bisect import bisect_left, insort


class Solution:
    def minimumSumSubarray(self, nums: list[int], l: int, r: int) -> int:
        n = len(nums)
```

```
        # prefix[i] = sum of the first i elements
        prefix = [0] * (n + 1)
        for i, num in enumerate(nums):
            prefix[i + 1] = prefix[i] + num

        window = []       # sorted prefixes whose window length is currently valid
        best = -1

        for end in range(l, n + 1):
            # the earliest start that keeps the window length >= l
            insort(window, prefix[end - l])

            # drop any start that would make the window longer than r
            if end - r - 1 >= 0:
                window.remove(prefix[end - r - 1])

            pos = bisect_left(window, prefix[end])
            if pos > 0:
                candidate = prefix[end] - window[pos - 1]
                if best == -1 or candidate < best:
                    best = candidate

        return best


def demo():
    sol = Solution()
    assert sol.minimumSumSubarray([3, -2, 1, 4], 2, 3) == 1
    assert sol.minimumSumSubarray([-2, -1], 1, 2) == -1
    assert sol.minimumSumSubarray([1, 2, 3, 4], 2, 4) == 3
    print("all checks passed")


if __name__ == "__main__":
    demo()
```

## Code Walkthrough

- `prefix` is built once, in a single pass, so `prefix[i]` always means "sum of the first `i` elements of `nums`."
- `window` holds every prefix value that's currently a legal starting point — meaning a subarray ending at `end` and starting there would have a length between `l` and `r`.
- `insort(window, prefix[end - l])` adds the newest legal start: at length exactly `l`, start index `end - l` just became valid.
- The removal step retires prefixes that have aged past `r` — once a window would exceed length `r`, that start is no longer usable.

- `bisect_left(window, prefix[end])` finds where today's prefix would sit in the sorted window. Anything before that index is strictly smaller than today's prefix.
- `window[pos - 1]`, the entry immediately before the insertion point, is the *largest* prefix smaller than today's — exactly the gap we want to minimize.
- `best` tracks the smallest valid positive sum found across the whole pass.

## Dry Run

Using `nums = [3, -2, 1, 4]`, `l = 2`, `r = 3`:

`prefix = [0, 3, 1, 2, 6]`

- `end = 2`: insert `prefix[0] = 0` → window `[0]`. `bisect_left(window, prefix[2]=1)` = 1. Candidate: `1 - 0 = 1`. `best = 1`.
- `end = 3`: insert `prefix[1] = 3` → window `[0, 3]`. Nothing to remove yet (`end - r - 1 = -1`). `bisect_left(window, prefix[3]=2)` = 1. Candidate: `2 - 0 = 2`. `best` stays `1`.
- `end = 4`: insert `prefix[2] = 1` → window `[0, 1, 3]`. Remove `prefix[0] = 0` (window would exceed length 3) → `[1, 3]`. `bisect_left(window, prefix[4]=6)` = 2. Candidate: `6 - 3 = 3`. `best` stays `1`.

Final answer: **1**, matching the example.

## Complexity Analysis

- **Binary search lookup:** `O(log n)` per step — this is correct and is the fast part.
- **Insertion/removal in a plain Python list** (`insort`, `list.remove`): each is `O(n)` because the underlying array shifts elements. Across `n` steps, that's `O(n²)` in the worst case.
- **Space:** `O(n)` for the prefix array and the sorted window.

So as written, the algorithm is a correct `O(n²)`-worst-case solution dressed in an `O(log n)`-lookup pattern — fine for this problem's constraint of `n ≤ 100`, where it's comfortably fast, but not yet the full `O(n log n)` the pattern promises. To actually hit `O(n log n)`, the sorted list needs to be backed by a structure with logarithmic insert and delete — a Fenwick tree indexed over compressed prefix values, a balanced BST, or a library implementation like `sortedcontainers.SortedList`. The algorithmic idea — prefix sums plus a sorted lookup — doesn't change; only the data structure underneath it does.

## Alternative Solutions

- **Fenwick tree / Binary Indexed Tree over compressed prefix values**: supports `O(log n)` insert, delete, and "largest value less than X" queries, giving true `O(n log n)`.
- `sortedcontainers.SortedList`: a drop-in replacement for the plain list that gives `O(log n)` insertion in practice (amortized), making the swap nearly free in terms of code changes.
- **Balanced BST (e.g., a treap or AVL-backed ordered set)**: conceptually the same guarantee as a Fenwick tree, more general but more code to maintain.

For `n ≤ 100`, none of this matters in practice — the plain-list version is simpler to write correctly under interview pressure, and that's a legitimate trade-off to state out loud if asked.

## Edge Cases

- **Empty array or `l` / `r` exceeding array length**: no valid window exists; should return `-1`.
- **All negative numbers**: every window sum is negative or zero; return `-1`.
- **All sums tie at the same positive value**: any of them is correct; the algorithm should return that value regardless of which window produced it.
- `l == r`: window length is fixed; the sorted window only ever holds one valid start at a time.
- **Values at the extremes (-1000 to 1000)**: prefix sums can swing widely; nothing breaks, since comparisons don't depend on magnitude, only relative order.

## Common Mistakes

1. Reaching for a sliding window and assuming shrinking always decreases the sum — false the moment negative numbers appear.
2. Forgetting to exclude zero — the problem asks for strictly positive sums, not non-negative ones.
3. Off-by-one errors in the prefix array indexing, especially around `end - l` and `end - r - 1`.
4. Removing the wrong prefix from the window — forgetting that the start being retired is tied to length `r`, not length `l`.
5. Assuming `bisect_left` plus a plain list gives `O(log n)` end-to-end, when the insertion/removal cost is the real bottleneck.
6. Initializing `best` to `0` instead of a sentinel like `-1`, which silently breaks the "no valid answer" case.

## Interview Questions

- "Why doesn't sliding window work here?" — be ready to explain the monotonicity argument concretely.

- "What's the actual time complexity of your solution, including the data structure operations, not just the binary search?"
- "How would you modify this to find the *maximum* sum below a limit, instead of the minimum positive sum?"
- "Could you solve this with a Fenwick tree instead of a sorted list? Walk through what changes."
- "What if `l` and `r` could be as large as 10^5 — how does your approach change?"

## Similar Problems

- **LeetCode 325 — Maximum Size Subarray Sum Equals k**: same prefix-sum-plus-hashmap pattern, different target condition.
- **LeetCode 209 — Minimum Size Subarray Sum**: sliding window works here because all values are positive, which is exactly the condition that breaks in problem 3364.
- **LeetCode 862 — Shortest Subarray with Sum at Least K**: prefix sums plus a monotonic deque, a close cousin of this problem's structure.
- **LeetCode 327 — Count of Range Sum**: prefix sums plus a merge-sort or BIT-based count, reinforcing the same "prefix difference lookup" idea at a different angle.

## Key Takeaways

When a subarray-sum problem includes negative numbers, check your sliding-window assumption before writing any code — it's often wrong. Prefix sums turn subarray sums into point-to-point differences, and once you're comparing differences, a sorted structure with binary search beats a linear scan. But watch the data structure doing the sorting: a correct algorithmic idea can still be slow if the container underneath has linear-time insertion. Know the gap between "the pattern is `O(n log n)`" and "my code is `O(n log n)`."

## Watch the Video

This walkthrough — including the full timing comparison between brute force and the optimized approach, and two quiz rounds to test your understanding — is available here: {LINK}

## About the Series

This breakdown is part of **Fun with Learning Technology**, a series where real LeetCode problems get traced from first instinct to working code — brute force first, then the reasoning that gets you to something faster, with actual runtime measurements instead of just complexity notation. The goal isn't

memorizing solutions; it's building the instinct to recognize these patterns the next time they show up in a different disguise.

## Call To Action

If this walkthrough helped the problem click, give the video a like on YouTube and subscribe for more problem breakdowns like this one. For deeper write-ups, dry runs, and the occasional bonus edge case that doesn't make it into the video, subscribe to the Substack. Drop a comment with how you'd have approached it, and share this with anyone prepping for interviews.

---

## Quick Review

**Pattern for 'min positive subarray sum' with +/- elements?**

Prefix sums + sorted search (or min-heap/multiset) to find smallest prefix[j]-prefix[i]>0; triggered by subarray-sum-range queries.

**Time complexity for prefix-sum + binary search approach?**

O(n log n): one pass to build prefix sums, each lookup via sorted structure insert/search is O(log n).

**Space complexity of the prefix-sum approach?**

O(n) for storing prefix sums in the sorted container (e.g. sorted list or multiset).

**Common bug when sums can be exactly zero?**

Zero isn't positive — must require strictly >0, so searching for prefix[i] you need the next value strictly greater, not equal.

**Minimum Positive Sum Subarray vs plain max-subarray (Kadane)?**

Kadane finds max sum in O(n); this needs smallest *positive* sum, which isn't monotonic, so Kadane's greedy drop doesn't apply directly.

**Interview follow-up: why does brute force (check all subarrays) fail at scale?**

$O(n^2)$ or $O(n^3)$ pairs/sums become too slow for large n (e.g. 1e5); prefix sums + sorted lookup gets it to O(n log n).

Minimum Positive Sum Subarray — free Python cheat sheet